F5 AI Security on Amazon EKS

This repository automates a single-node Amazon EKS proof of concept for F5 AI Security Guardrails.

The PoC uses:

- Amazon EKS with one GPU managed node group
- F5 AI Security Operator installed from Harbor
- Guardrails inference enabled
- Red Team optional and disabled by default
- In-cluster PostgreSQL
- F5 NGINX Ingress Controller for NGINX Open Source
- Optional Route 53 DNS management
- Self-signed TLS for lab access

Route 53 is optional. If you do not use Route 53, the script still deploys the environment and prints the ingress load balancer hostname so you can create DNS manually with another provider.

Architecture

```
Browser
  |
  | https://guardrails.f5demo.io
  v
DNS record
  |
  v
AWS Network Load Balancer
  |
  v
NGINX Ingress Controller
  |
  +-- /auth -> cai-moderator:8080  (Keycloak)
  |
  +-- /     -> cai-moderator:5500  (F5 AI Security UI/API)
```

Application Namespaces

Namespace	Purpose
`f5-ai-sec`	F5 AI Security Operator
`cai-moderator`	Guardrails UI/API, Keycloak auth, PostgreSQL
`prefect`	Prefect server, worker, workflow registration
`f5-ai-sec-inference`	Guardrails inference service and scanner model
`nginx-ingress`	F5 NGINX Ingress Controller and AWS load balancer

Sizing Note

The F5 runbook recommends more headroom for single-node deployments. This PoC uses `g5.4xlarge` as the smallest practical AWS GPU instance because it provides:

- 1 NVIDIA A10G GPU with 24 GiB VRAM

- 16 vCPU
- 64 GiB memory

For Guardrails only, `g5.4xlarge` worked in this PoC. For Guardrails and Red Team together, the F5 runbook calls for dedicated GPUs per product:

- Guardrails: 1 GPU with at least 24 GB VRAM
- Red Team: 1 GPU with at least 48 GB VRAM

That means both products together need two GPUs. On AWS, use an instance family/size that presents at least two suitable GPUs and enough CPU/memory headroom, such as an A10G multi-GPU size for PoC testing or an A100/H100 class node where available. A single-GPU `g5.4xlarge` is not enough for both Guardrails and Red Team.

Prerequisites and Preflight

The script checks prerequisites before deployment. You do not need to manually run every AWS and Kubernetes verification command unless troubleshooting.

Required local tools:

- `aws`
- `kubectl`
- `helm`
- `eksctl`
- `openssl`
- `python3`
- `curl`

Required inputs:

- AWS CLI credentials with permissions for EKS, EC2, IAM, CloudFormation, ELB, EBS, and Route 53
- Route 53 hosted zone for the selected hostname, only if `ROUTE53_ENABLED=true`
- Harbor registry credentials for `harbor.calypsoai.app`
- F5 AI Security license string
- LLM provider credentials for post-install app configuration

Configure AWS CLI:

```
aws configure
```

Run preflight:

```
./scripts/guardrails-poc.sh preflight
```

The preflight check validates local tools, AWS identity, region, optional Route 53 settings, GPU instance availability, GPU quota visibility, Harbor login, license input, and `eksctl` cluster configuration.

Script Deployment

Use the script for normal PoC lifecycle operations.

```
./scripts/guardrails-poc.sh preflight
./scripts/guardrails-poc.sh up
./scripts/guardrails-poc.sh status
./scripts/guardrails-poc.sh down --yes
```

The script is idempotent. If deployment stops partway through, rerun `up`; it reuses existing AWS and Kubernetes resources and continues from the current state.

Configure Inputs

Copy the example config:

```
cp config/guardrails-poc.env.example config/guardrails-poc.env
```

Edit `config/guardrails-poc.env` if you need to change region, cluster name, hostname, hosted zone, instance type, Red Team, Route 53, ingress, or secret file paths.

Default secret paths:

```
HARBOR_CREDENTIALS_FILE="config/harbor.txt"
F5_LICENSE_FILE="config/license.txt"
```

Important feature toggles:

```
ROUTE53_ENABLED="false"
ENABLE_RED_TEAM="false"
```

When Route 53 is disabled, the script still creates the ingress load balancer and prints its hostname. You can then create DNS manually with any DNS provider. Set `ROUTE53_ENABLED=true` to let the script honor `ROUTE53_ZONE_NAME` and `ROUTE53_HOSTED_ZONE_ID`.

When Red Team is enabled, make sure the license covers Red Team and the node group has enough dedicated GPU capacity.

Relative paths are resolved from the repository root.

Deploy

```
./scripts/guardrails-poc.sh up
```

Expected runtime is roughly 25-45 minutes for a new environment because EKS control plane, GPU node, add-ons, images, model pod, ingress load balancer, and DNS all need time to come online.

At the end, the script prints a concise status summary with component health, UI URL, initial login credentials, and password-change links.

Status

```
./scripts/guardrails-poc.sh status
```

Example:

```
Guardrails PoC status
=====================
Environment:               running
Public URL:                online (https://guardrails.f5demo.io)
DNS:                       present -> <load-balancer>
Ingress LB:                ready -> <load-balancer>
EKS cluster:               ACTIVE, Kubernetes 1.35, region us-east-1
Node:                      1/1 Ready, type g5.4xlarge
Storage:                   ready (EBS CSI active, gp2 default)
f5-ai-security-operator:   ready (1/1 replicas)
cai-moderator:             ready (2/2 pods)
f5-ai-sec-inference:       ready (2/2 pods)
prefect:                   ready (3/3 pods)
Red Team:                  disabled
nginx-ingress:             ready (1/1 replicas)
EBS leftovers:             none found
```

Stop and Delete

```
./scripts/guardrails-poc.sh down --yes
```

This deletes Route 53 records when enabled, the EKS cluster, GPU node group, load balancer, EBS volumes created through the cluster, available leftover cluster-owned EBS volumes, and `eksctl` CloudFormation stacks.

First Login and Password Change

Initial credentials from the F5 runbook:

```
Username: admin
Password: pass
```

The Guardrails UI account page does not expose password management. Use the embedded Keycloak account console.

For this PoC, change the password in the `master` realm:

```
https://guardrails.f5demo.io/auth/realms/master/account/#/security/signingin
```

The deployment can also expose a `calypsoai` realm:

```
https://guardrails.f5demo.io/auth/realms/calypsoai/account/#/security/signingin
```

Keycloak passwords are realm-specific. If more than one realm accepts the initial password, rotate it in each realm.

Manual Deployment Reference

The script is the recommended path. Use this section only if you want to understand or manually reproduce what the script does.

1. Create EKS

```
eksctl create cluster \
  --name f5-guardrails-poc \
  --region us-east-1 \
  --version 1.35 \
  --managed \
  --nodegroup-name guardrails-gpu-ng \
  --node-type g5.4xlarge \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1 \
  --node-volume-size 150 \
  --node-ami-family AmazonLinux2023 \
  --install-nvidia-plugin \
  --vpc-nat-mode Disable
```

2. Prepare Storage

```
eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name eks-pod-identity-agent \
  --force

eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name aws-ebs-csi-driver \
  --auto-apply-pod-identity-associations \
  --force

kubectl patch storageclass gp2 \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

3. Install Operator and Guardrails

```
kubectl create namespace f5-ai-sec

kubectl create secret docker-registry regcred \
  --docker-server='harbor.calypsoai.app' \
  --docker-username='<HARBOR_USERNAME>' \
```

```
  --docker-password='<HARBOR_PASSWORD>' \
  -n f5-ai-sec

helm registry login harbor.calypsoai.app

helm upgrade --install f5-ai-security-operator \
  oci://harbor.calypsoai.app/calypsoai/f5-ai-security-operator-helm \
  --version 1.4.1 \
  -n f5-ai-sec
```

Apply a `SecurityOperator` custom resource with:

- `registryAuth.existingSecret: regcred`
- in-cluster PostgreSQL enabled
- `CAI_MODERATOR_BASE_URL` set to `https://guardrails.f5demo.io`
- `CAI_MODERATOR_DEFAULT_LICENSE` set to your F5 license string
- Guardrails inference enabled
- Red Team enabled only if your license and GPU capacity support it

4. Complete Supporting Services

The first manual deployment required these fixes, which are now automated by the script:

- Wait for Prefect API health before recreating the workflow registration job.
- Install F5 NGINX Ingress Controller with an internet-facing AWS NLB.
- Create a self-signed TLS secret for `guardrails.f5demo.io`.
- Create an ingress routing `/auth` to port `8080` and `/` to port `5500` on `cai-moderator`.
- Optionally create a Route 53 CNAME from `guardrails.f5demo.io` to the ingress load balancer.

Troubleshooting

Use the script summary first:

```
./scripts/guardrails-poc.sh status
```

Useful detailed commands:

```
kubectl get pods -A
kubectl get svc -n nginx-ingress nginx-ingress-controller -o wide
kubectl get ingress -n cai-moderator -o wide
kubectl logs -n cai-moderator deploy/cai-moderator --tail=100
kubectl logs -n f5-ai-sec deploy/controller-manager --tail=100
kubectl logs -n f5-ai-sec-inference deploy/f5-ai-sec-inference --tail=100
kubectl logs -n prefect deploy/prefect-server --tail=100
```

If Route 53 is enabled and the public URL is not reachable immediately after the record is updated, wait a few minutes for DNS propagation and local resolver cache refresh. If Route 53 is disabled, create a DNS record manually that points your hostname to the ingress load balancer.